

ProCon-V: A Programmable Tightly Coupled Convolution Accelerator Based on RISC-V Custom Instructions for Edge Devices

Ahmed Kamaleldin , *Member, IEEE*, Habib Aouinti , and Diana Göhringer , *Member, IEEE*

Abstract—The increasing use of deep neural network (DNN) based applications on edge devices has driven the need for innovative computing solutions that can meet the high computational requirements of DNN workloads while ensuring low power consumption. Therefore, enhancing the general-purpose (GP) computing units of edge devices by integrating DNN-specific functions and operations into existing instruction set architectures (ISAs) provides the flexibility to support a wide variety of neural network models, thereby improving computing efficiency. This article proposes a programmable tightly coupled accelerator for convolution operations called ProCon-V. It is based on RISC-V custom instructions, extending the standard ISA to support 2-D convolution operations. The accelerator is tightly coupled with a RISC-V-based pipeline through an open-source extension interface [21], offloading convolution operations from the pipeline's main execution unit. The ProCon-V computing engine is based on a scalable systolic array architecture for partial-sum computation, with hardware-supported Im2Col transformation that supports multi-channel convolution layers with various configurations. Moreover, the tightly coupled accelerator enables the execution of convolution layers through a set of custom instructions handled by the RISC-V core without requiring extra modifications to the core pipeline. ProCon-V is implemented and evaluated on an AMD/Xilinx Virtex Ultrascale+ FPGA featuring dual clock domains for the RISC-V core and the convolution accelerator. Evaluation results demonstrate that the proposed accelerator features a maximum computing throughput of 174.4 GOPS for INT8-based operations with an energy efficiency of 181.7 GOPS/W.

Index Terms—RISC-V, ISA extension, CNN, neural networks accelerators, FPGA, edge computing.

I. INTRODUCTION

THE widespread adoption of DNN-based applications on edge devices has created a growing need for novel computing methodologies that address the high computational demands of DNN workloads while maintaining low power consumption. Nowadays, dedicated DNN accelerators are embedded on edge devices for inference workloads offloading from GP processing cores, aiming to enhance computing scalability and efficiency [1]. To meet the computational needs of edge devices, DNN accelerators are customized according to target applications requirements and operating conditions.

Therefore, the state-of-the-art adopts custom hardware accelerators based on several sorts of computing architectures (e.g., systolic arrays, single-instruction multiple-data (SIMD)) with a wide range of dataflow management techniques [2]. Also, quantization has been introduced as an algorithmic optimization technique in order to reduce memory footprint and area/hardware resources utilization. As a result, quantized neural network (QNN) models are currently deployed on resource-constrained edge devices computing units (e.g., ARM, RISC-V-based microcontrollers) [3]. With the emergence of lightweight NN models (e.g., MobileNet, YOLO), there is an increasing need for flexible and configurable hardware accelerators capable of supporting various operations at runtime to meet the requirements of different model workloads. Several versatile DNN accelerators such as NVDLA [4], AMD/Xilinx DPU [5], and Google TPU [6] have been released, where they support a wide range of NN models with different types of operations. Versatile DNN accelerators are commonly used in heterogeneous computing systems as standalone accelerators connected to GP cores as memory-mapped peripherals through standardized interconnect fabrics (e.g., AXI, PCIe). This type of DNN accelerator is based on a scalable number of computing units dedicated to common NN layers (e.g., convolution, pooling, and fully connected). They are configured based on the requirements of the target neural network model at the kernel or layer level, which often results in lower utilization rates and increased power consumption for lightweight NN models.

On the other hand, extending GP cores with ISAs allows the flexibility to tune and configure NN operations for a broader range of NN models according to the NN layer operations

Received 22 December 2024; revised 31 October 2025; accepted 1 February 2026. Date of publication 11 February 2026; date of current version 9 April 2026. This work was supported in part by the German Research Foundation ("Deutsche Forschungsgemeinschaft") (DFG) under Grant Project-ID 287022738 TRR 196 for Project S05, and in part by the German Federal Ministry of Education and Research BMBF as part of the DAKORE (Datenfunknetz mit Adaptivhardware und KI-Optimierung zur Reduktion des Energieverbrauchs) Project under Grant 16ME0433K. Recommended for acceptance by P. Zhou. (Corresponding author: Ahmed Kamaleldin.)

The authors are with the Chair of Adaptive Dynamic Systems, TU Dresden, 01187 Dresden, Germany (e-mail: ahmed.kamal@tu-dresden.de; habib.aouinti@mailbox.tu-dresden.de; diana.goehringer@tu-dresden.de).

Digital Object Identifier 10.1109/TC.2026.3663365

requirements [7]. Also, this approach can reduce the hardware resources overhead and power consumption by increasing the reutilization of computing units among different modes of operation for a NN layer (e.g., convolution, pooling, fully connected layers) [8], [9]. However, most of ISA-based NN accelerators rely on closed-source instruction set architectures, which restrict their extensibility and hinder seamless integration into different heterogeneous computing systems. In this work, we present a programmable tightly coupled convolution accelerator called ProCon-V that extends the RISC-V ISA to efficiently support convolution operations across diverse CNN models. The accelerator is managed and configured via a set of custom instructions, enabling the offloading of convolution operations from the main RISC-V core execution unit to the accelerator's computing units. Our contributions in this work rely on the following:

- Design and implementation of a programmable convolution accelerator based on custom RISC-V ISA. The accelerator features a tightly coupled connection based on the open source XIF interface [21] with the CV32e40X core to offload convolution operations from the pipeline's execution unit.
- Enable convolution layers execution via custom instructions, with unified memory access between the core and the accelerator through the core data port (load/store unit).
- Support for multi-kernel, multi-channel 2D convolution layers with variable stride configurations, using on-the-fly hardware-supported Im2Col transformation optimized for streaming access patterns between the accelerator's on-chip memory and compute unit.

The rest of the article is structured as follows: Section II discusses background and related work. The tightly coupled accelerator design approach and microarchitecture of ProCon-V are introduced in Section III. Evaluation and experimental results are presented and analyzed in Section IV. Finally, Section V concludes this work.

II. RELATED WORK

A wide range of convolutional neural network (CNN) accelerators has been developed to improve the computational efficiency of CNN inference on edge computing platforms. These accelerators are typically implemented either as standalone custom accelerators or as hardware extensions coupled with GP computing units to offload the inference workload. The choice of coupling technique significantly influences the degree of programmability, configurability, and interaction between the accelerator and the host processor [26]. In a loosely coupled architecture, the CNN accelerator is implemented as a memory-mapped peripheral, interfaced with the GP computing unit via on-chip interconnect (e.g., AXI or AHB). In this setup, accelerator configuration and control are managed through software APIs, and special care has to be taken to handle data coherency and synchronization between the processor and the accelerator [10]. In contrast, tightly coupled accelerators are implemented as custom pipeline extensions directly connected to the host processor datapath [26]. In this model, configuration, control,

and data movement are performed through custom instructions executed by the processor, enabling a fine-grained interaction with the accelerator. This approach allows configuration, controlling, and data management to be handled through custom instructions executed by the GP computing unit. This section provides a review of related work on programmable and configurable CNN accelerators with different coupling techniques with GP compute units.

A loosely coupled configurable CNN accelerator is proposed by Shao et al. [11] for remote sensing applications. The accelerator is a lightweight accelerator based on a systolic array architecture for FPGA devices. A micro instruction controller is used to control and configure the accelerator through predefined instructions generated by an external host PC. The controller serves to manage loading/storing operations as well as configure the systolic array size according to layer parameters. Also, Sim et al. architecture [12] features a micro instruction controller to manage and configure an energy-efficient CNN inference accelerator. It supports different voltage/frequency domains for computing and memory management with a scalable number of compute engine units.

To support different types of convolution operations (i.e., depth-wise (DW) and standard convolutions), Amoeba [13] is proposed as a hybrid CNN accelerator with an arbitrary kernel size and optimized dataflow for different layer configurations. The accelerator can be classified as a standalone accelerator with an internal control system for managing the scheduling of layer operations and the dataflow between compute units and storage systems. It supports heterogeneous compute modules based on a custom 2D MAC array. In a similar context, SNAP [14] is proposed as a sparse CNN accelerator for different types of convolution layers with a custom PE architecture for sparse compute operations. SNAP features scalable compute units for a high level of parallelization with an internal control unit for compute units configuration and dataflow management. Despite the high performance gain of the previous standalone accelerators, the degree of programmability and a seamless way of controlling are limited. Several recent co-design approaches have leveraged the flexibility and extensibility of the RISC-V ISA by coupling various CNN accelerators with the RISC-V pipeline through custom instruction extensions. The primary goal is to seamlessly integrate CNN functionality into the processor pipeline as macro-operations, which can be invoked using custom instructions. This approach enhances programmability, reduces software overhead, and minimizes the need for complex control routines typically required in loosely coupled accelerator architectures. Therefore, CONNA [15] is proposed as a configurable matrix multiplication engine controlled by a RISC-V-based processor for NN acceleration. The communication between the accelerator and the processor is conducted through the Rocket co-processor interface (RoCC) using custom instructions for controlling and configuration. CONNA's computing engine is based on configurable vector lanes for different modes of operations that can be configured according to the shape and dimension of the target matrix multiplication. Similarly, Gemmini [16], [25] is proposed as a DNN accelerator tightly coupled

TABLE I
STATE-OF-THE-ART COMPARISON WITH LOOSELY AND TIGHTLY COUPLED RISC-V-BASED CNN ACCELERATORS

Related Work (Year)	Core	Coupling Technique ^a	Interfacing	Memory model ^b	Pipeline Modifications	Programming method	Prec.	Platform
CONNA [15] (2022)	Rocket	TC	RoCC + TileLink interconnect	Shared memory ^c	None	Custom Instructions	8-bit	ASIC
Gemmini [16], [25] (2019)	Rocket	TC	RoCC + TileLink interconnect	Shared memory ^c	None	Custom Instructions	8-bit	ASIC
EWS [17] (2024)	zero RISCY [24]	LC	AXI interconnect	Shared memory ^c	None	APIs/Drivers	8-bit	ASIC
Chen et al. [19] (2024)	Custom RISC-V core	TC	Pipeline custom extension	Direct access to off-chip memory	Decoding stage	Custom Instructions	8-bit	ASIC
Wang et al. [18] (2023)	Custom RISC-V core	TC	Pipeline custom extension + AXI interconnect	Shared memory ^c	Decoding and execution stages	Custom Instructions	8-bit	FPGA
This Work	CV32e40X	TC	CV-X-IF	Unified memory access via the core data port	None	Custom Instructions	8-bit	FPGA

^aTightly coupled (TC): The accelerator is directly connected to the host core pipeline as a co-processor unit. Loosely coupled (LC): The accelerator is connected to the host core as a memory-mapped peripheral. ^bMemory model describes how the accelerator shares the data with the RISC-V core data memory. ^cThe RISC-V core and the accelerator have shared access to the core memory through AXI or TileLink interconnect.

with a RISC-V-based processor, enabling a hardware/software co-design approach for DNN workloads. The main compute engine is based on different spatial array architectures for MAC operations with local scratchpad memory banks. Additionally, Gemmini features an optional hardware-supported Im2Col unit [25], offloading the data reshaping task from the processor and thereby reducing software overhead. Both CONNA and Gemmini adopt a shared memory model between the processor and the accelerator through the TileLink interconnect. On the other hand, EWS [17] is proposed as an energy-efficient CNN accelerator integrated with a RISC-V-based SoC as a loosely coupled unit through an AXI interconnect. The accelerator features an internal DMA unit for data and instruction transfer between the internal L1 memory and the SoC L2 main memory. It supports an enhanced weight stationary (WS) dataflow between local memory and the compute systolic array to improve data reusability during operation.

Wang et al. [18] proposed a reconfigurable CNN accelerator controlled by custom RISC-V instructions, featuring separate control and data interfaces. The data interface is connected to external DRAM via the AXI interconnect, while the control interface is integrated into the decode and execution stages of the RISC-V pipeline to manage operations without a dedicated decoder inside the accelerator. Similarly, Chen et al. [19] introduced a flexible dataflow accelerator with the control unit connected after the decode stage, using custom instructions to configure layer parameters and dataflow. However, both designs require essential modifications to the target RISC-V core to be tightly coupled with the pipeline.

Accordingly, our proposed tightly coupled CNN accelerator is integrated with the open-source CV32e40X RISC-V core through the XIF extension interface without extra modifications or control unit insertion to the core pipeline. In contrast, proposed works by [18], [19] rely on closed-source cores with custom pipeline extensions that require extra pipeline

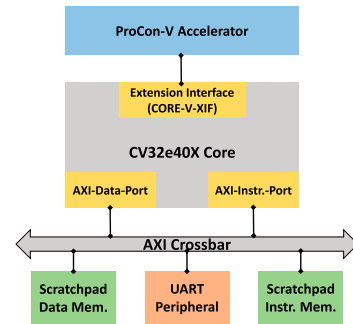


Fig. 1. Overall SoC architecture of the proposed convolution accelerator tightly coupled with the CV32e40X core.

modifications, which limit the degree of portability and seamless integration with accelerators. All custom instructions are issued through the XIF and decoded autonomously by the accelerator. In addition, both control and data interfaces are directly connected to the core pipeline through the extension interface, and the accelerator leverages the core load/store unit (LSU) to access the core data memory as shown in Fig. 1. This results in a unified memory access and ensures data coherency between the accelerator and the RISC-V core, reducing shared memory management complexity. A detailed comparison with related RISC-V-based CNN accelerators, highlighting coupling technique, memory model, and programming method, is provided in Table I, emphasizing our contribution in enabling convolution layer execution via custom RISC-V instructions without requiring any modifications to the RISC-V core pipeline.

III. PROCON-V MICROARCHITECTURE

A. Architecture Overview

In this work, the open-source CV32e40X core [20] is the selected RISC-V core which supports the integration of custom execution units or co-processor-based architectures

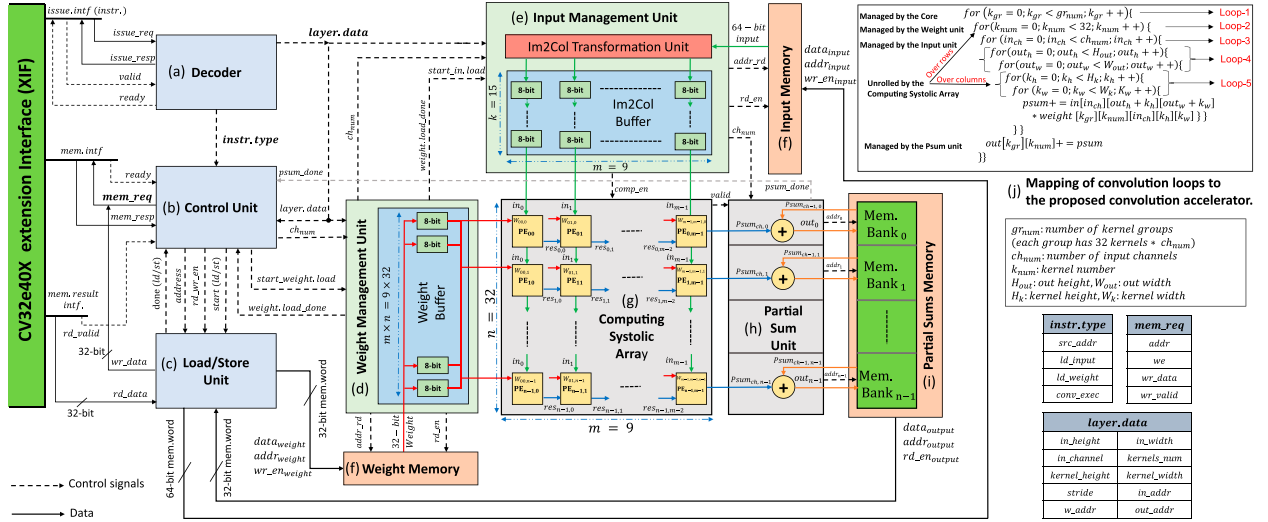


Fig. 2. Overview of the proposed tightly-coupled convolution accelerator and mapping method of convolution loops.

through its extension interface (CV-X-IF, shortly XIF) [21]. Therefore, it provides the possibility to offload the execution of custom instructions to custom execution units without modifying the core pipeline. As shown in Fig. 1, the proposed convolution accelerator is tightly coupled with the CV32e40X core through the XIF. In our case, the convolution accelerator operates as a RISC-V-based co-processor. The CV32e40X core is a 32-bit 4-stage in-order pipeline that implements the RV32 ISA. The complete SoC consists of a single core extended by the proposed convolution accelerator, scratchpad data/instruction memories, and a UART peripheral, which are connected through an AXI crossbar. The proposed accelerator is composed of the following main units, as shown in Fig. 2:

- 1) **Decoder and control units:** the decoder unit, as shown in Fig. 2(a), is responsible for receiving and identifying the functionality of custom instructions offloaded by the RISC-V core. The decoder sends several configuration parameters extracted from custom instruction payloads to the control unit, as shown in Fig. 2(b). Based on the decoded instruction and received configurations, the control unit manages the convolution operations and load/store (ld/st) data over the accelerator.
- 2) **Load and store unit:** it manages the data transfer between the core data memory and the accelerator local scratchpad memories for weight, input, and partial sum (Psum)/output feature map (OFM) as shown in Fig. 2(c). According to the received custom instructions, the control unit issues the control signals (i.e., address, read/write enable) to the ld/st unit. The ld/st unit is connected to the memory result interface of the core extension port in order to directly load input/weight values to the local accelerator memories, and manage the write-back/store operation of the output values to the core data memory.
- 3) **Input and weight management units:** they are responsible for managing the dataflow between scratchpad memories for input and weight and the

systolic array as shown in Fig. 2(d) and (e). The input unit implements on-the-fly Im2Col transformation to convert convolution operations into a vector matrix multiplication over the systolic array. The Im2Col transformation supports kernel sizes up to 3×3 and stride values of 1 and 2. The weight unit manages the transmission of the weight values to the systolic array PEs. Input and weight units are designed to handle 8-bit integer values supporting the WS dataflow model.

- 4) **Computing systolic array and partial sum manager:** the computing array is based on a systolic array architecture of 32×9 PEs as shown in Fig. 2(g). Each PE implements a MAC unit for vector-matrix multiplication operations between input and weight kernels. Weight kernels values are stored internally within the PE weight register file. The systolic array is designed to store weight values of a maximum of 32 kernels of size 3×3 , where each PE store a single weight value. Therefore, the systolic array is capable of executing all vector-matrix multiplication operations for 32 kernels (size 3×3) in parallel. In total 288 MAC operations can be conducted at the same time. The systolic array output represents $32 \times \text{Psum}$ values ($\text{Psum}(k_{num})$) of $32 \times \text{kernels}$ for each input channel as described by Eq. (1)¹.

$$\begin{aligned}
 & \text{Psum}(k_{num})[in_{ch}][out_h][out_w] \\
 &= \sum_{k_h=0}^{H_k-1} \sum_{k_w=0}^{W_k-1} in[in_{ch}][out_h + k_h][out_w + k_w] \\
 & \quad \times weight(k_{num})[in_{ch}][k_h][k_w]
 \end{aligned} \quad (1)$$

$$\begin{aligned}
 & out[k_{num}][out_h][out_w] \\
 &= \sum_{in_{ch}=0}^{c_{num}-1} \text{Psum}[in_{ch}][k_{num}][out_h][out_w]
 \end{aligned} \quad (2)$$

¹Equations parameters are described in Fig. 2(j).

The Psum unit, as shown in Fig. 2(h), is attached to the systolic array to generate the final OFM for multi-channel input layers as described by Eq. (2)¹. The Psum unit can conduct up to 32 addition operations in parallel for $32 \times \text{Psum}$ values from $32 \times \text{kernels}$.

The proposed convolution accelerator handles 2-D convolution loops as shown in Fig. 2(j). For each input channel, the systolic array can handle up to 32 kernels of size 3×3 in parallel by unrolling loop-2 over 32 rows of $9 \times \text{PEs}$ each. Loop-5 is unrolled by a factor of 9 over each row to generate Psum values for each input channel according to Eq. (1)¹. Input channels are handled sequentially by loop-3, where input and weight units feed the next input channel and the corresponding weight kernels to the systolic array. The output is generated by accumulating the Psum values of all input channels by the Psum unit within loop-3. In case of convolution layers with a high number of kernels per input channel (more than 32 kernels), the execution is conducted by splitting the computation based on several kernel groups of $32 \times \text{kernels}$ each as described by loop-1 Fig. 2(j). Accordingly, the accelerator generates part of the OFM corresponding to each kernel group. The RISC-V core manages loop-1 by sequentially sending the subsequent kernel groups to the accelerator to obtain the complete OFM of the executed convolution layer.

B. Custom Instructions and Tightly-Coupled Connection

Custom instructions are issued by the CV32e40X core to the convolution accelerator through the extension interface issue port as shown in Fig. 2(a). The issue interface is responsible for connecting the core decoding stage to the accelerator decoder unit. The issue interface offloads custom instructions to the accelerator through the (*issue_req*) signal to specify the type of the instruction to be executed followed by an (*issue_resp*) signal from the accelerator. In our case, all supported custom instructions are 32-bit uncompressed instructions based on the RV32I ISA with custom opcodes. Custom instructions are based on the standard RISC-V 'R-type' format as shown in Fig. 3 with two source register files (rs1, rs2) and a destination register (rd). A single custom opcode of "0001011" and (funct3, funct7) fields are used to identify the four supported custom instructions as follows:

- 1) **src_addr**: it is responsible for loading/setting input and weight start addresses (*in_addr*, *w_addr*) into the accelerator parameter registers "*layer.data*" to be used later during data loading/storing operations. The instruction stores the input start address value in the rs1 register and the weight start address in the rs2 register as shown in Fig. 3. Upon instruction execution, the contents of rs1 and rs2 are loaded to accelerator parameter registers. The rd register is not used by this instruction, and (funct3, funct7) fields are set to zeros.
- 2) **ld_weight**: it is responsible for loading weight values from the core data memory to the accelerator weight memory. A single instruction calling can load up to $32 \times \text{kernels}$ of size 3×3 for all channels to the weight memory. Also, the instruction is used to load/set

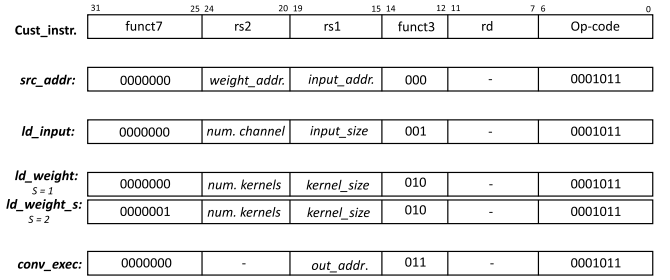


Fig. 3. The proposed RISC-V-based custom instructions.

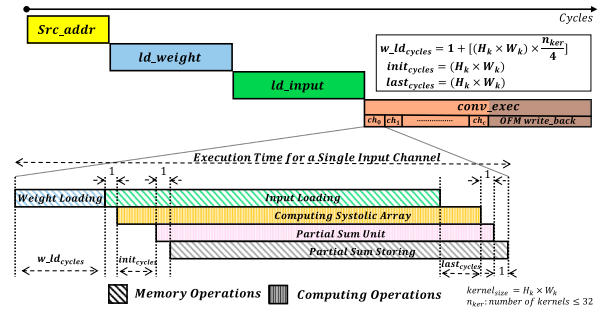


Fig. 4. Timing diagram of the operations sequence to execute a single convolution layer showing the execution order of custom instructions.

(number of kernels and kernel size) to the accelerator parameter registers "*layer.data*". As shown in Fig. 3, the rs1 and rs2 source registers include the kernel size (H_k, W_k) and number of kernels associated with the number of channels (ch_{num}, n_{ker}) respectively. They are used to determine the data size for the weight loading operation. There are two categories of this instruction to support two stride size configurations of ($S=1, S=2$) determined by Funct7 field. The rd register is not used by this instruction and funct3 field is set to "010".

- 3) **ld_input**: it is responsible for loading input values from the core data memory to the accelerator input memory. A single instruction calling can load all input channels to the input memory. Also, the instruction is used to load/set (input size and number of channels) to the accelerator parameter registers "*layer.data*". As shown in Fig. 3, the rs1 and rs2 source registers include the input size and number of input channels respectively. They are used to determine the data size during input loading operation. The rd register is not used by this instruction and funct3 field is set to "001".
- 4) **conv_exec**: it is responsible for starting the execution of convolution operations for the loaded weight and all input channels based on layer parameters stored in "*layer.data*" registers. After the execution of all input channels, the instruction handles the transfer of OFM to the core data memory as shown in Fig. 4. As shown in Fig. 3, the rs1 source register sets the output start address to the core data memory. The rd register is not used by this instruction and funct3 field is set to "011".

As shown in Fig. 2, the XIF memory and result interfaces are used for memory request/response signals and data transfer. They connect the accelerator control and load/store units to the core load store unit (LSU). The memory interface is responsible for handling all memory access between the accelerator and core data memory. The accelerator control unit (shown in Fig. 2(b)) is responsible for sending memory request signals, including source and destination addresses, to the core data memory as well as associated valid and read/write enable signals through the core LSU. During input and weight loading processes, the control unit manages the accelerator load/store unit (Fig. 2(c)) for data transfer between accelerator local memories and core data memory. The memory result interface is responsible for data transactions from the core data memory to the accelerator input and weight memories during loading instructions. For the write-back operation, the memory interface through the “*mem_req*” is used to transfer the OFM from the Psum memory to core data memory as *write_data* is part of the memory request signal.

Fig. 4 shows the sequence of operations based on the proposed custom instructions to execute a whole convolution layer. Initially, source addresses have to be loaded through *src_addr* instruction for input and weight, followed by input and weight load instructions (*ld_input*, *ld_weight*). The *conv_exec* initiates the execution process of the target convolution layer for each input channel. The execution process consists of five operations that are executed simultaneously in a pipeline fashion as shown in Fig. 4. Those five operations are completely managed by the accelerator without direct control from the core side. Three types of memory operations are conducted for data transactions between accelerator local memories and compute units: (a) the weight loading operation requires (w_ld) cycles to load (n_{ker}) kernels for a single input channel to the systolic array PEs as shown by cycle counts in Fig. 4, (b) the input loading operation for loading a single input channel values to the input management unit, and (c) storing Psum values for each input channel to the Psum memory. The computation process starts one clock cycle after the start of the input loading operation, waiting for the availability of the first valid input data to the systolic array. Afterwards, initial clock cycles (*init_cycles*) have to be consumed in order to receive the first array of Psum values, followed by an extra one clock cycle delay prior to storing the Psum values to the Psum memory. For subsequent input values, the computing and memory operations are handled in a fully pipelined way. After the execution of all input channels, a write-back operation as the second part of *conv_exec* is managed by the load/store unit to store the OFM values to the core data memory.

C. Dataflow Management

The dataflow management is conducted by weight and input management units as shown in Fig. 2(d) and (e). Those units are responsible for managing data transfer from input/weight memory to the systolic array. Prior to the computation process, all input channels and the corresponding weight parameters are stored completely in the input and weight memories (Fig. 2(f)).

The weight dual-port memory has a 32-bit word. Each memory word consists of 4×8 -bit integer weight parameters. The control unit issues a “*start_weight.load*” signal to the weight management unit associated with the corresponding channel number to start the weight loading operation of up to $32 \times$ kernels to the systolic array PEs. The weight unit converts kernel matrices of size $(H_k \times W_k)$ into column vectors of size $(1 \times (H_k \times W_k))$ and stores them accordingly in a weight buffer of a size of 32×9 locations. The weight loading operation requires $(\frac{n_{ker} \times H_k \times W_k}{4})$ clock cycles to load all weight parameters from the weight memory to the weight buffer and a single clock cycle to transmit them to all active systolic array PEs as shown in Fig. 4. As the weight stationary dataflow is supported by the accelerator, each PE stores a single weight parameter value to remain stationary during the computation. Each row of the systolic array architecture is responsible for handling MAC operations for a single weight kernel of a size up to 3×3 , where each PE stores a single parameter (8-bit value) of the assigned kernel.

Afterwards, a “*weight.load_done*” signal is sent to the control unit and the input management unit as a start signal to initialize computing operations by starting loading input data from the input memory to the input management unit, as shown in Fig. 4. The input dual-port memory has a 64-bit word. The input memory stores 8×8 -bit input values in a single address location. Initially, all input channels data are loaded from the core data memory to the input memory through the *ld_input* instruction. Where the load/store unit concatenates each consecutive two 32-bit words from the core data memory into a single 64-bit word and stores it into the input memory. The input management unit implements a custom on-the-fly streaming Im2Col transformation that converts convolution operations into vector-matrix multiplication operations. This transformation is necessary to increase the utilization of systolic array PEs during computation. The input management unit acts as a bridge between the input memory and the systolic arrays by fetching input data from the memory and transforming it into a specific format for vector-matrix multiplication through the Im2Col unit as shown in Fig. 2(e). By reorganizing input data into columns, the Im2Col allows the unrollement of Loop-5 over the systolic array PEs for up to 32 kernels. The custom Im2Col offers a low-latency transformation process for large input data sizes by splitting the input data into several input tiles. The tiling process is row-based, starting with the first input data row and moving horizontally till the end of the row and then moving to the subsequent rows for the complete input channel size. The input tile has a height equal to the kernel height H_k and a configurable width of one or two consecutive memory words based on the stride parameter. For example, in case of a kernel size of 3×3 and $S = 1$, the tile size is equal to 3×8 as shown in Fig. 5(a). As the input memory has a 64-bit data width, a single input tile row of (8×8) -bit can be fetched every clock cycle to the input management unit. The Im2Col unit is responsible for reading a single input tile from the input memory and sending the result of the transformation to the Im2Col buffer.

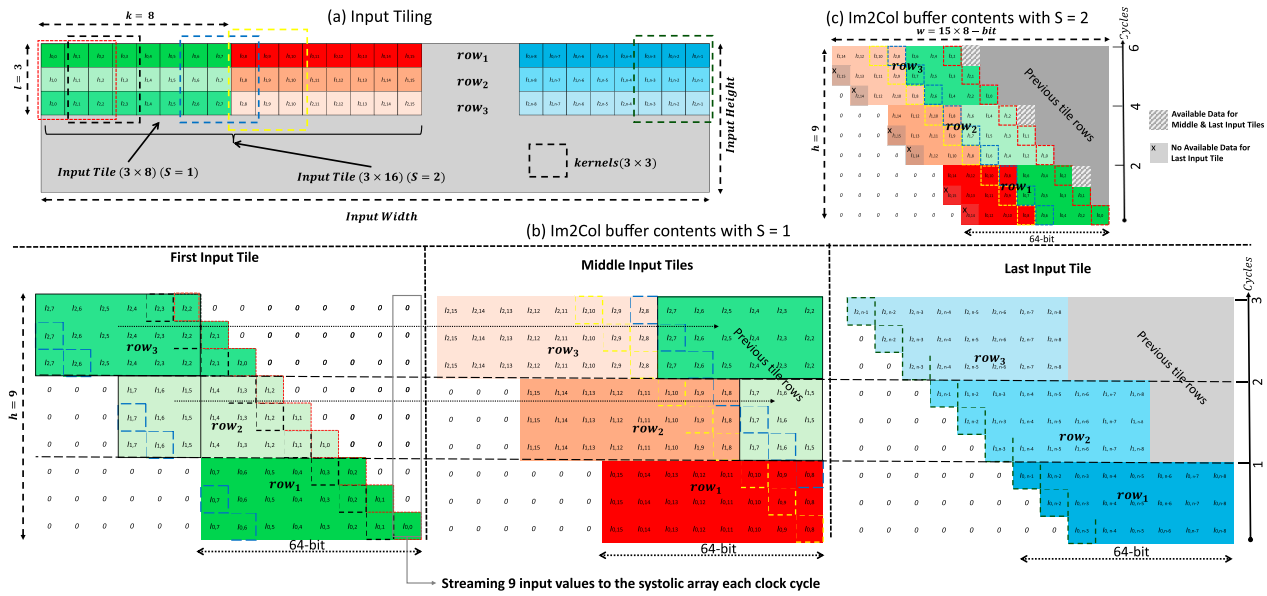


Fig. 5. A description of the on-the-fly streaming Im2Col transformation for a 3×3 kernel size showing (a) the input tiling approach, (b) Im2Col buffer contents with $S = 1$, and (c) Im2Col buffer contents with $S = 2$.

The buffer stores a single transformed input tile and feeds it to the systolic array PEs according to the correct order of input values during computation. The converted input data (the input columns corresponding to kernel size) are stored in a diagonal way in the Im2Col buffer as shown in Fig. 5(b) and 5(c) to correctly feed input data to the systolic array PEs for the vector-matrix multiplication operations (Eq. (1)). The Im2Col buffer has a fixed size of (9×15) locations where each location stores a single input value of 8 bits. This fixed size is suitable to store input tiles of 3×8 and 3×16 for the two supported stride configurations. As a result of the fixed input tile width of eight, the input channel width must be divisible by eight. To ensure this, the core inserts additional columns of zeros before sending the input channels to the accelerator. The Im2Col transformation converts a single input tile into multiple specific shapes according to the stride value and the input tile placement within the 2-D input channel matrix. In the case of $S = 1$, there are three states that determine the transformation shape of the tile according to the tile placement as shown in Fig. 5(b). The first state is when the tile is at the beginning of the input channel row for the first kernels. The second state handles the middle tiles and ensures the continuation of input data feeding. The third state is for the last tile, which ensures the correctness of the last diagonal kernels. Similarly, in the case of $S = 2$, three transformation shapes are required for the first, middle, and last input tiles as shown in Fig. 5(c). The Im2Col buffer streams the input data to the systolic array PEs in each single clock cycle. The buffer is implemented as a two-dimensional FIFO capable of receiving and transmitting data simultaneously. Fetching and transforming 8 input values (a single tile row) requires a single clock cycle in the case of $S = 1$ and two clock cycles in the case of $S = 2$. For example, in the case of a kernel size of 3×3 , three clock cycles are required to

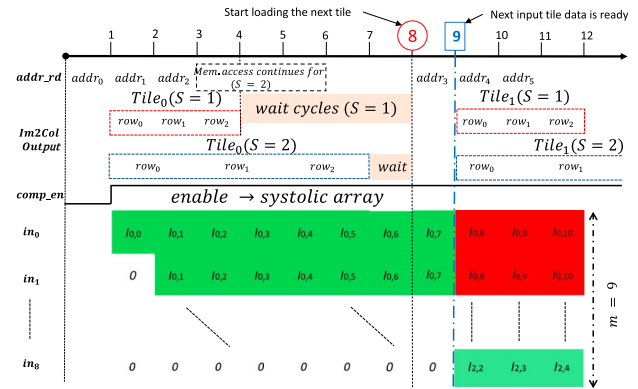


Fig. 6. A timing diagram showing the Im2Col transformation process (the input to the systolic array is shown for the case of $S = 1$ and a 3×3 kernel size).

transform and store a single input tile to the Im2Col buffer as shown in Fig. 5(b). Fewer clock cycles are consumed for smaller or one-dimensional kernel sizes as the tile size gets smaller, which requires less memory access.

As shown in Fig. 6, the transformation process starts by sending the required address to the input memory, loading the first row during the first clock cycle. The Im2Col unit is capable of loading a new input tile from the memory and storing the previously transformed tile in the buffer on the same clock cycle. Therefore, the systolic array starts the computation process one clock cycle after the input management unit loads the first tile row from the input memory. Accordingly, the systolic array processes new input data received from the Im2Col buffer in each single clock cycle. To sustain continuous data streaming to the systolic array, the Im2Col unit fetches a new input tile every $8 \times \text{clock}$ cycles as shown at the top of Fig. 6. The Im2Col unit

requires four clock cycles to fill the buffer with the transformed input tile in case the kernel size is 3×3 , while the systolic array requires nine clock cycles to process the current Im2Col buffer data in case of a 3×3 kernel. Therefore, the Im2Col unit has fixed wait cycles for both stride values according to the kernel size to ensure proper data streaming to the systolic array as shown in Fig. 6. The number of wait clock cycles depends on the number and size of rows in each input tile, which in turn are determined by the kernel width (W_k) and the stride value. The proposed dataflow management with the custom on-the-fly streaming Im2Col transformation enables the systolic array to perform convolution operations as vector-matrix multiplication between the unrolled input channel data and the flattened weight values, thereby increasing the computational efficiency.

D. Computing Engine

The systolic array represents the core computational engine of the proposed accelerator. It is designed to accelerate vector-matrix multiplication operations, particularly for convolution layers. The systolic array features a 2-D array of PEs, where each PE implements a MAC unit. The 2-D array consists of 32×9 PEs as shown in Fig. 7, where each row consists of 9 PEs to execute the required vector-matrix multiplication operations of a single kernel of a size up to 3×3 each clock cycle. The systolic array is capable of handling the computation of max 32 kernels in parallel with 32 rows of PEs. The Psum values are generated by the systolic array PEs, where a maximum of 32 Psum values can be generated each clock cycle. The systolic array operates on 8-bit signed integer for input and weight values while resulting in a 16-bit signed integer for Psum values. The internal architecture of a single PE is shown in Fig. 7. It consists of a single 8-bit multiplier and a single 16-bit adder to realize the MAC unit. For all PEs in the first systolic array column, the result is obtained directly from multiplication, without requiring accumulation from previous PEs. Also, a register file to store a single weight parameter during the weight loading phase, which is enabled by the *ld_weight_en* signal supporting the weight stationary dataflow model. The PE is responsible for forwarding input values to the subsequent PE in the same column and propagating its output result *res* to the next PE in the same row to generate Psum values. According to the target kernel size and number of kernels, specific columns and rows of the systolic array are active during computation as shown in Fig. 7. Therefore, each PE has a PE_{en} signal that determines whether to forward the output of the MAC operation ($PE_{en} = 1$) or bypass the previous output result ($PE_{en} = 0$) to the next PE.

The computing operation starts by receiving the enable signal from the input management unit (*comp_en*), indicating the availability of input data in the Im2Col buffer. The systolic array processes up to nine input values streamed from the Im2Col buffer each clock cycle, as shown in Fig. 7. The kernel size ($H_k \times W_k$) is responsible for determining the number of active columns and from which column the final Psum value is valid. For example, in the case of a 3×3 kernel size, all 9 PEs per row are active, and the ninth PE output result is connected to

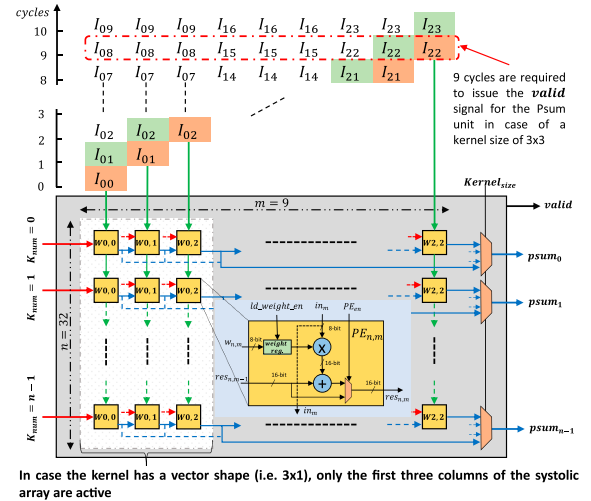


Fig. 7. A schematic overview of the computing systolic array, and the PE internal architecture.

this row's Psum port. In case of a vector kernel size (i.e., 3×1), only 3 PEs are active as only three weight parameters are stored per row and three input values are fed from the Im2Col buffer. Therefore, the Psum port is re-connected to the output result of the third PE as shown in Fig. 7. Similarly, if the number of kernels is fewer than 32, only the corresponding rows of PEs with available weight parameters will remain active. The Psum unit receives Psum values from the systolic array to perform the accumulation process for all input channels. Initially, the systolic array sends a valid signal to the Psum unit, indicating the arrival of the first Psum values from all active rows. The valid signal is generated after a specific number of clock cycles from the systolic array's initial clock cycle, where the delay duration depends on the number of active PEs per row. For example, with a 3×3 kernel size, a delay of 9 clock cycles is needed to issue the valid signal. In contrast, for a 3×1 vector kernel size, only 3 clock cycles are required to produce the first Psum values from all active PE rows.

The input management unit sends the processed channel number (ch_{num}) to the Psum unit, indicating which input channel data is fed to the systolic array. The Psum unit has an internal register file that stores the size of the OFM calculated from *layer.data* signals sent by the decoder unit to determine the data size to be stored in the memory bank and for address generation. For the first input channel ($ch_{num} = 0$), the Psum unit directly stores the received Psum values from the systolic array to the memory banks, bypassing the accumulation units. For subsequent input channels, the Psum unit accumulates the Psum values from previous input channels to the current Psum values, generating the corresponding OFM according to Eq. (2). The complete Psum unit includes 32 computing blocks (Psum block) to accumulate the Psum values for each weight kernel separately. Therefore, each Psum block generates a single OFM channel ($out[k_{num}][H_{out}][W_{out}]$), where k_{num} represents the output channel number, which corresponds to the systolic array connected row number. The internal architecture of the Psum block and the associated dual-port memory bank are shown

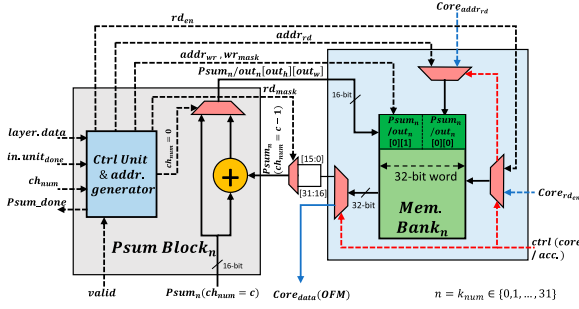


Fig. 8. A schematic overview of the Psum block internal architecture and the associated dual-port memory bank.

in Fig. 8. The Psum block consists of two main parts, a 16-bit signed adder for Psum accumulation and a control unit to manage data transfer with the corresponding Psum memory bank. Psum blocks start upon receiving the *valid* signal from the active systolic array rows, where the number of active Psum blocks is equal to the number of active rows. The dual-port memory bank has two separate ports for reading and writing Psum values during the accumulation process. Also, the read port is connected to the accelerator's load/store unit to read the final OFM results for all processed input channels, as shown by the memory bank schematic in Fig. 8. The main control unit of the accelerator has to configure the memory bank read interface to be connected either to the Psum block during the Psum accumulation or to the load/store unit during the write-back operation.

During the Psum accumulation, the inner control unit is responsible for controlling the flow of data between the Psum block and memory bank ports by generating the required memory addresses and read/write enable signals. The memory bank has a word size of 32-bit and stores two Psum values at the same address location. Accordingly, during the write operation, a write mask signal (*wr_mask*) is used to select the correct 16-bit location within the memory address location. Similarly, a read mask signal (*rd_mask*) is used to read the corresponding 16-bit Psum value from the previous input channel. The Psum accumulation process has a latency of two cycles, one cycle is used to read the previous input channel Psum values from the memory and do the accumulation, while the second cycle is required to write the result back to the same memory location. After all input channels are fed to the systolic array, the input unit sends a done signal to the Psum unit, indicating the end of the convolution operation. Accordingly, the Psum unit issues a *Psum_done* signal to the main control unit to initiate the write-back operation, during which all OFM channels are loaded to the core data memory. During the write-back operation, two OFM values (2×16 -bit) are stored at each address location of the core data memory.

E. High Level Programming Method

The accelerator is fully controlled by the CV32e40X core through a software-based C function as shown in Fig. 9. The C function calls the custom instructions and sets their

```

1  int8_t weight_conv[], input_conv[]; int32_t out_conv
   [], dummy;
2  void conv_layer(uint8_t S, uint16_t k_h, uint16_t k_w,
3    uint16_t in_w, uint16_t in_h, uint16_t num_channels,
4    uint16_t num_kernels) {
5    uint32_t in_size = (in_w << 16) | in_h;
6    uint32_t ch_num = (num_channels << 16);
7    uint32_t ker_size = (k_w << 16) | k_h;
8    uint32_t n_ker = (num_kernels << 16) | num_kernels;
9    uint32_t *w_addr = (uint32_t *) &weight_conv[0];
10   uint32_t *in_addr = (uint32_t *) &input_conv[0];
11   uint32_t *out_addr = (uint32_t *) &out_conv[0];
12   int gr_num = (num_kernels) / 32;
13   for (int i = 0; i < gr_num; ++i) { //Loop-1 in Fig. 2
14     if (gr_num > 1) { n_ker = (num_channels << 16) | 32;
15       w_addr += 32*k_h*k_w*num_channels;
16       out_addr += 128*in_h*in_w; }
17     asm volatile (//Send weight, input source addr.
18       "src_addr %[z], %[x], %[y]\n\t"
19       : [z] "=r" (dummy)
20       : [x] "r" (w_addr), [y] "r" (in_addr) );
21     if (S == 2) { asm volatile (//weight loading S=2
22       "ld_weight_s %[z], %[x], %[y]\n\t"
23       : [z] "=r" (dummy)
24       : [x] "r" (ker_size), [y] "r" (n_ker)); }
25     else if (S == 1) { asm volatile (//weight loading S=1
26       "ld_weight %[z], %[x], %[y]\n\t"
27       : [z] "=r" (dummy)
28       : [x] "r" (ker_size), [y] "r" (n_ker)); }
29     if (i == 0) { //Input channels loading at first iter
30       asm volatile (
31         "ld_input %[z], %[x], %[y]\n\t"
32         : [z] "=r" (dummy)
33         : [x] "r" (in_size), [y] "r" (ch_num) ); }
34     asm volatile (//Conv. execution and OFM write-back
35       "conv_exec %[z], %[x], %[y]\n\t"
36       : [z] "=r" (dummy)
37       : [x] "r" (out_addr), [y] "r" (out_addr)); }
38   }

```

Fig. 9. Software-based C function for handling convolution layers using custom instructions executed by the RISC-V core.

parameters according to the target layer configurations. This function serves as an abstraction layer, bridging the gap between high-level software functions running by the core and the low-level custom instructions that control the accelerator.

This step of software support requires the insertion of the four supported custom instructions into the RISC-V GNU toolchain [22] to be recognized during the compilation of the C-based function. The software function implements the convolution loops described in Fig. 2(j), where the first loop is managed by the core (Line-13) to iterate over several kernel groups of 32 kernels. The rest of the convolution loops for each kernel group are managed completely by the accelerator through custom instructions. Therefore, the software function manages the execution of large convolutional layers with more than 32 kernels per channel by splitting them into multiple kernel groups that can be handled sequentially by the core. Initially, weight and input load instructions are executed sequentially (Line-22, 31) followed by the *conv_exec* instruction to accelerate the execution of the corresponding kernel group on the accelerator. Accordingly, this method provides the flexibility to manage and configure the accelerator from the software side, enabling seamless integration of the accelerated convolution function with the other software-based functions running on the core.

IV. IMPLEMENTATION AND EVALUATION RESULTS

A. FPGA Implementation

ProCon-V accelerator has been developed and implemented using a modular and hierarchical design approach. Where architectural modules (i.e., systolic array, input/weight units, memory blocks, etc.) are implemented as intellectual property (IP) components to be integrated together to build the convolution accelerator. The implementation is realized at the RTL level using Verilog and SystemVerilog. In this work, AMD/Xilinx Virtex Ultrascale+ XCVU9P is the target FPGA for implementation, prototyping, and evaluation of the proposed accelerator. Also, Vivado ML 2023.2 is used for RTL synthesis, simulation, place and routing, and final bitstream generation for testing.

The tightly coupled accelerator with the CV32e40X core is integrated within a bus-based SoC architecture as described by Fig. 1. The core and SoC peripherals are running at a clock frequency of 150 MHz, whereas the proposed accelerator has dual clock domains of 150 MHz and 300 MHz. The accelerator's decoder, control, and load/store units are running at the same clock frequency of the core, as they are managing the communication between the core and the accelerator through the XIF. On the other hand, the input/weight, Psum managers, and the systolic array are running at 300 MHz to increase the computing performance. Both weight and input memories are implemented as a dual-clock dual-ported memory where the write port is connected to the slow clock domain and the read port to the fast clock domain. The Psum memory is implemented as a dual-port memory with a single clock port connected to the 300 MHz clock domain. This enables the Psum unit to perform read and write operations during the accumulation process at 300 MHz, while the core accesses the memory only to read the final OFM values from the slow clock domain. Table II shows the hardware resource utilization of the accelerator, the core, and the complete SoC components. The accelerator consumes ($\sim 81\%$) of the total LUTs utilized by the SoC. On-chip URAM blocks are used during implementation for input, weight, and Psum memories. In total 105 URAM blocks are used, where ($\sim 91\%$) of them are consumed by the Psum memory. The systolic array consumes 288 DSP blocks, where each PE consumes a single DSP block to realize the MAC unit. The rest of DSP blocks are partially used by control and management units over the accelerator. Furthermore, the power consumption of the accelerator and the complete SoC are estimated by Vivado power estimation.

B. Computing Performance Evaluation

In this work, the proposed convolution accelerator is evaluated based on: (1) data transfer latency between core data memory and accelerator scratchpad memories during load and store operations, (2) computing performance of the accelerator in terms of integer operations per cycle (Ops/cycle), achievable memory bandwidth (bytes/cycle), and execution time. The evaluation is conducted over several sets of 2-D convolution layers from three different QNN models provided by AMD/Xilinx FINN examples [23], and a custom set of convolution layers.

TABLE II
HARDWARE RESOURCE UTILIZATION AND ESTIMATED POWER
CONSUMPTION TARGETING AMD/XILINX VIRTEX ULTRASCALE+
(XCVU9P) FPGA

Units	Resources				Dynamic Power (mW)
	LUTs	FFs	URAMs	DSPs	
SoC (Total)	28522	20835	248	313	1029
Data Mem.	634	375	128	0	
Instr. Mem.	360	295	15	0	
AXI Crossbar	619	421	0	0	
Compute Tile	26909	19744	105	313	989
CV32e40X	3744	2352	0	3	29
Conv. Acc.	23165	17392	105	310	960
Control Unit	71	32	0	0	
Decoder	9	232	0	0	
ld/st Unit	2292	145	0	0	
Input Unit	4533	2920	0	17	
Weight Unit	3577	4753	0	4	
Psum Unit	1014	71	0	1	
Systolic Array	10413	9189	0	288	
Psum Mem.	1152	0	96	0	
Input Mem.	62	27	6	0	
Weight Mem.	42	23	3	0	

Estimated Static Power = 2466 mW, for the complete SoC.

Total Estimated Power = Static + Dynamic power = 3495 mw

The first set of convolution layers is based on the quantized RadioML (VGG10) model, all layer parameters are shown in Table III. Those layers feature a vector shape for both input and kernel sizes. The second set of convolution layers is based on selected layers from the quantized MobileNet-v1 and ResNet-50 models, all layers parameters are shown in Table V. The layers dimensions are shown based on the ONNX format where the input size is represented by $(ch_{num} \times H_{out} \times W_{out})$, and the weight size is represented by $(k_{num} \times ch_{num} \times H_k \times W_k)$. The selected layer from the MobileNet-v1 model has a high number of kernels and a low number of input channels. In contrast, the selected layers from the ResNet-50 model feature a high number of both kernels and input channels. The third set of convolution layers is based on custom convolution layers with different numbers/sizes of kernels and different input sizes. The goal of using those custom layers is to evaluate the accelerator computing performance, especially with layers that have a low number of kernels (below 32). As a result, the utilization of the systolic array varies depending on the number of kernels per layer. All custom layer parameters are shown in Table IV. For all computing performance evaluation, the core clock cycle of 150 MHz is considered to calculate latency, Ops/cycle, bytes/cycle, and the accelerator computing throughput (GOPS). Fig. 10 shows the accelerator execution time and data transfer latency, both measured per cycle for the three convolution sets. The execution time for each convolution set corresponds to the latency of the *conv_exec* instruction, excluding the write-back stage. It represents the latency of the convolution accelerator to fully execute a convolution layer without including data transfer latency between the accelerator and core data memory. The data transfer latency consists of two parts, the loading latency and the storing latency. The loading latency reflects the number of clock cycles needed to transfer input and weight data from the core data memory to the

TABLE III
CONVOLUTION SET 1 PARAMETERS: RADIOML (VGG10) CONVOLUTION LAYERS

Layers	Size	Input Size	Weight Size	Output Size	Layers	Size	Input Size	Weight Size	Output Size
CONV1, S = 1		$2 \times 1024 \times 1$	$32 \times 2 \times 3 \times 1$	$32 \times 1024 \times 1$	CONV5, S = 1		$32 \times 64 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 64 \times 1$
CONV2, S = 1		$32 \times 512 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 512 \times 1$	CONV6, S = 1		$32 \times 32 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 32 \times 1$
CONV3, S = 1		$32 \times 256 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 256 \times 1$	CONV7, S = 1		$32 \times 16 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 16 \times 1$
CONV4, S = 1		$32 \times 128 \times 1$	$32 \times 32 \times 3 \times 1$	$32 \times 128 \times 1$					

TABLE IV
CONVOLUTION SET 3 PARAMETERS: CUSTOM CONVOLUTION LAYERS

Layers	Size	Input Size	Weight Size	Output Size	Layers	Size	Input Size	Weight Size	Output Size
CONV1, S = 1		$1 \times 32 \times 1$	$1 \times 1 \times 3 \times 1$	$1 \times 32 \times 1$	CONV7, S = 1		$3 \times 56 \times 56$	$20 \times 3 \times 3 \times 3$	$20 \times 56 \times 56$
CONV2, S = 1		$1 \times 32 \times 1$	$32 \times 1 \times 3 \times 1$	$32 \times 32 \times 1$	CONV8, S = 1		$10 \times 16 \times 16$	$10 \times 10 \times 3 \times 3$	$10 \times 16 \times 16$
CONV3, S = 1		$1 \times 1024 \times 1$	$32 \times 1 \times 3 \times 1$	$32 \times 1024 \times 1$	CONV9, S = 1		$10 \times 16 \times 16$	$20 \times 10 \times 3 \times 3$	$20 \times 16 \times 16$
CONV4, S = 1		$3 \times 16 \times 16$	$10 \times 3 \times 3 \times 3$	$10 \times 16 \times 16$	CONV10, S = 1		$10 \times 16 \times 16$	$32 \times 10 \times 3 \times 3$	$32 \times 16 \times 16$
CONV5, S = 1		$3 \times 16 \times 16$	$20 \times 3 \times 3 \times 3$	$20 \times 16 \times 16$	CONV11, S = 1		$10 \times 1024 \times 1$	$32 \times 10 \times 3 \times 1$	$32 \times 1024 \times 1$
CONV6, S = 1		$3 \times 56 \times 56$	$10 \times 3 \times 3 \times 3$	$10 \times 56 \times 56$					

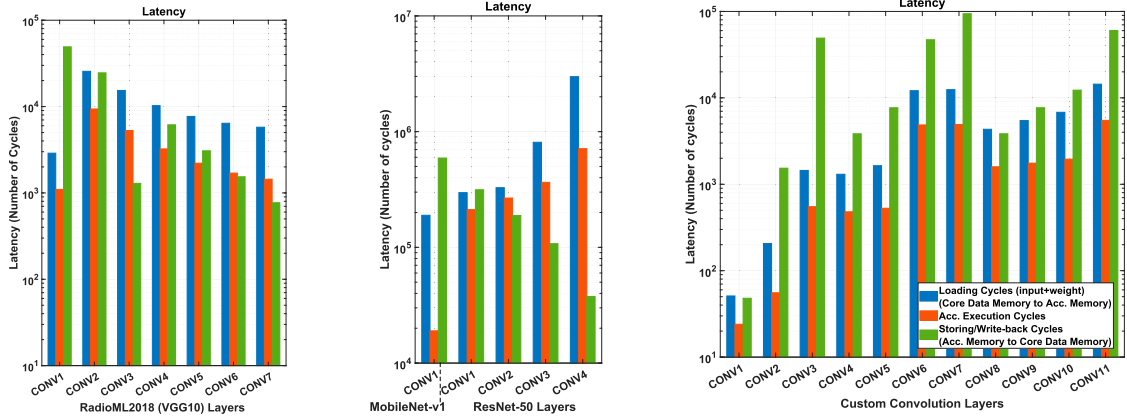


Fig. 10. Data transfer latency between the core data memory and the convolution accelerator scratchpad memories, and convolution accelerator execution time for different sets of convolution layers (red bars).

TABLE V
CONVOLUTION SET 2 PARAMETERS: SELECTED CONVOLUTION LAYERS FROM MOBILENET-V1 AND RESNET-50 MODELS

MobileNet-v1				
Layers	Size	Input Size	Weight Size	Output Size
CONV1, S = 2		$3 \times 224 \times 224$	$32 \times 3 \times 3 \times 3$	$32 \times 111 \times 111$
ResNet-50				
CONV1, S = 1		$64 \times 56 \times 56$	$64 \times 64 \times 3 \times 3$	$64 \times 56 \times 56$
CONV2, S = 1		$128 \times 28 \times 28$	$128 \times 128 \times 3 \times 3$	$128 \times 28 \times 28$
CONV3, S = 1		$256 \times 14 \times 14$	$256 \times 256 \times 3 \times 3$	$256 \times 14 \times 14$
CONV4, S = 1		$512 \times 7 \times 7$	$512 \times 512 \times 3 \times 3$	$512 \times 7 \times 7$

accelerator's input and weight memories. The storing latency reflects the latency of the write-back operation to transfer the OFM to the core data memory. As shown in Fig. 10, loading and storing latency are related to the input/output dimension as well as the number of kernels for each convolution layer. With small input dimensions, like the case of the RadioML model and the custom convolution set, the loading and storing operations consume more cycles than the computing operations due to the smaller number of computing operations and

the higher operating frequency of the convolution accelerator compared to the core. On the other hand, for ResNet-50, the storing operations consume fewer cycles in comparison with the accelerator execution time due to the high number of computing operations conducted by the systolic array and the Psum unit. A detailed computing performance evaluation based on the accelerator execution time (red bars in Fig. 10) for the three convolution sets is presented in Tables VI–VIII. For all models, the workload size indicates the number of multiplication and addition operations required by each layer. In addition, the memory transaction size indicates the amount of read and write data between the accelerator's weight, input and Psum memories and the systolic array during the accelerator execution time. The achievable throughput (Ops/cycle) for each convolution layer varies depending on the kernel size, number of kernels, and input size. The workload size is calculated based on the required MAC (mul+add) and Psum add operations for each layer. Since the RadioML convolution layers use a 3×1 kernel, as shown in Table III, only one-third of the systolic array PEs are active during computation. As a result, the achievable throughput is approximately 30% of that of MobileNet-v1

TABLE VI
ACCEL. COMPUTING PERFORMANCE EVALUATION ACROSS CONV. LAYERS FROM RADIOML (VGG10) MODEL (CONV. SET 1)

Metrics \ Layers	CONV1	CONV2	CONV3	CONV4	CONV5	CONV6	CONV7
Workload (KOP) ($\text{Ops} \times 10^3$)	425	3654	1827	913	457	228	114
Mem. Transaction (KBytes)	198.9	2084.8	1044.5	524.2	264.2	134.1	69.1
Throughput (Ops/cycle)	388	389	346	283	207	135	79
Mem. Bandwidth (Bytes/cycle)	181	222	198	162	120	79	48
Execution Time (μs)	7.32	62.5	35.2	21.5	14.7	11.3	9.6

The reference clock cycle is the core cycle of 150 MHz. The core cycle period = $2 \times$ accelerator cycle period.

TABLE VII
ACCEL. COMPUTING PERFORMANCE EVALUATION ACROSS SELECTED LAYERS FROM MOBILENET-V1 AND RESNET-50 (CONV. SET 2)

Model \ Layers	MobileNet-v1	ResNet-50				
Metrics	CONV1	CONV1	CONV2	CONV3	CONV4	
Workload (MOP) ($\text{Ops} \times 10^6$)	22.1	243.8	243.9	244	244	
Mem. Transaction (MBytes)	4.4	51.5	51.5	52	53.7	
Throughput (Ops/cycle)	1162	1151	915	669	342	
Mem. Bandwidth (Bytes/cycle)	231	243	193	143	75	
Execution Time (μs)	126.66	1412.28	1778.37	2430.34	4751.41	

The reference clock cycle is the core clock cycle of 150 MHz. The core cycle period = $2 \times$ accelerator cycle period.

and ResNet-50 models. Additionally, for RadioML layers with small input sizes, a further reduction in computing throughput occurs due to the shift from compute-dominated operations to memory-dominated operations. As shown in Fig. 4, no compute operations are performed during the weight loading cycles until all weights have been transmitted to the PEs. Consequently, for layers with fewer number of computing operations, a reduction in throughput can occur. This can be shown by small custom convolution layers (i.e., CONV1, CONV2) in Table VIII. In cases where the number of kernels is large, the weight load phase consumes a significant number of cycles, thereby reducing the achievable throughput of the target layer. As shown in Table VII, ResNet-50 layers with a high number of kernels in conjunction with a high number of input channels (i.e., CONV3 and CONV4) exhibit an achievable throughput that is approximately 60% and 30% of the throughput of the CONV1 layer respectively, which has 64 kernels per channel. Custom convolution layers with a lower number of kernels (i.e., CONV6, CONV8) exhibit lower throughput than layers with 32 kernels (i.e., CONV10), as shown in Table VIII. This is due to fewer PEs being utilized for layers with a lower kernel number. Fig. 11 shows the roofline model of the proposed accelerator based on the three convolution sets. The maximum throughput is achieved by the MobileNet-v1 layer due to the high number of computing operations in comparison with memory operations. RadioML achieves the lowest throughput due to the unutilized PEs during computation. Due to the higher number of weight loading cycles, the throughput of ResNet-50 layers is inversely proportional to the number of kernels per layer. For custom layers with fewer than 32 kernels, the throughput decreases due to inactive PEs in the systolic array.

To evaluate the overall performance of the extended RISC-V core with the proposed accelerator, within the SoC architecture

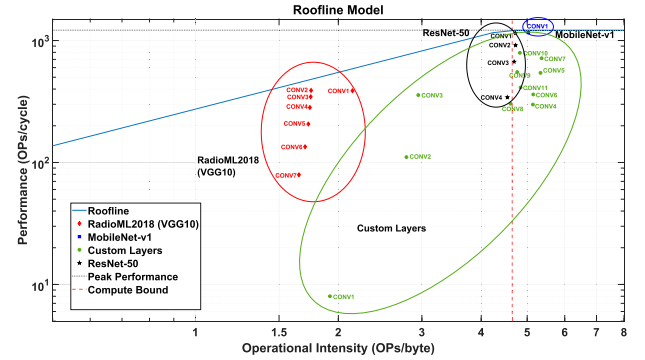


Fig. 11. The roofline model of the convolution accelerator.

shown in Fig. 1. The total execution time is measured for both the baseline core and the extended core with the accelerator across all convolution sets as shown in Fig. 12. The total execution time is measured by taking into account the accelerator computing/execution time (red bars in Fig. 10) and the data transfer latency for loading and storing inputs, weights, and OFMs between the core data memory and the accelerator scratchpad memory (green and blue bars in Fig. 10). The achieved speedup is influenced by the ratio between computation time and data transfer latency. In layers where the computation dominates (i.e., the ratio of accelerator execution time to data transfer latency is high), the speedup is significantly high, up to $2200\times$ for ResNet-50 Conv1, Conv2 layers. In contrast, when data transfer latency becomes a larger portion of total execution time, the speedup is relatively lower due to the limited memory bandwidth between the core and data memory. This is because both the baseline and the extended core access the same core data memory through the CV32e40X

TABLE VIII
ACCEL. COMPUTING PERFORMANCE EVALUATION ACROSS CUSTOM CONV. LAYERS (CONV. SET 3)

Metrics \ Layers	CONV1	CONV2	CONV3	CONV4	CONV5	CONV6	CONV7	CONV8	CONV9	CONV10	CONV11
Workload (KOP)	0.2	6.2	196.6	143.4	286.7	1756.1	3512.3	483.8	967.7	1548.3	2261
Mem. Trans. (KBytes)	0.1	2.2	66.7	27.9	53.8	341.1	654.9	104.9	203.1	320.9	1278.8
Throughput (Ops/cycle)	8	111	357	299	543	361	716	303	551	793	412
Mem. BW (Bytes/cycle)	4	40	121	58	102	70	134	66	116	164	233
Exec. Time (μ s)	0.16	0.37	3.67	3.2	3.52	32.4	32.72	10.63	11.71	13.01	36.6

KOP: Ops $\times 10^3$. The reference clock cycle is the core clock cycle of 150 MHz. The core cycle period = 2 $\times$ accelerator cycle period.

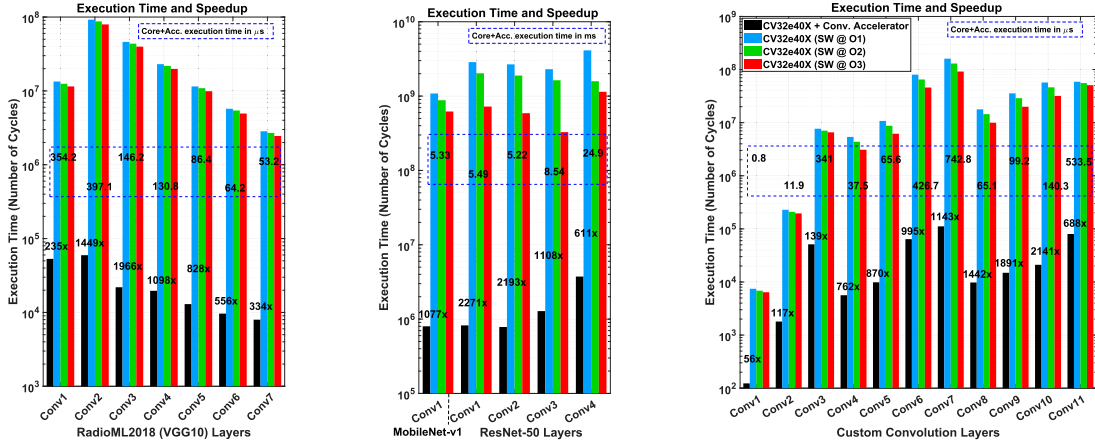


Fig. 12. The total execution time and achieved speedup of the CV32e40X core extended with the accelerator compared to only the baseline CV32e40X core within the SoC shown in Fig. 1. The total execution time is measured, taking into consideration the accelerator execution time and data transfer latency from/to the core data memory and the accelerator scratchpad memory.

LSU. Therefore, performance gains are mainly attributed to the accelerator execution time.

C. Comparison With State-of-the-Art

The proposed ProCon-V accelerator has been compared with several recent SoA accelerators in terms of various design and performance metrics. Our evaluation covers two aspects: (1) the accelerator computing performance based on the accelerator computation/execution cycles (red bars in Fig. 10) without including memory latency overhead between RISC-V core data memory and the accelerator scratchpad memory (loading and storing cycles in Fig. 10); and (2) The whole SoC performance shown in Fig. 1 based on the total execution time (Fig. 12), including the accelerator execution cycles and loading/storing cycles for loading and storing IFMs, weights, and OFMs between the core data memory and the accelerator scratchpad memory. In addition, resource utilization and power consumption of both the accelerator and the full SoC, including the accelerator, RISC-V core, and data/instruction memory, as shown in Table II. In contrast, some SoA designs report evaluation results based solely on the accelerator computing performance, without considering extra latency overhead introduced by the RISC-V/host core for control, or the data transfer latency between on-chip/off-chip memory and the accelerator. Tables IX, X show the comparison of our proposed approach with ASIC-based and FPGA-based SoA CNN accelerators respectively. All considered

SoA accelerators support 8-bit to 16-bit QNN models. Our convolution accelerator is based on a systolic array computing engine for parallel vector-matrix multiplication operations with 288 MAC units. As shown in Table IX, ProCon-V accelerator supports a higher number of MAC units in comparison with [14], [15], [16], [19]. Moreover, the accelerator features two clock domains, where the systolic array and the associated dataflow management units are running at twice the clock frequency of the RISC-V core. In comparison with SoA mentioned in Table IX, our proposed accelerator achieves a maximum computing performance of 1162 Ops/cycle relative to the clock cycle of the RISC-V core. This computing performance is based only on the accelerator execution time (the execution time of the *conv_exec* instruction in Fig. 4) without including data loading/storing latency (*src_addr*, *ld_input*, *ld_weight*, and *write_back* in Fig. 4) between the core data memory and the accelerator scratchpad memory. In contrast, the full SoC, which includes both accelerator computing and memory latency overhead, achieves a maximum computing performance of 311 Ops/cycle. This reduction is due to the data transfer latency overhead incurred during the loading and storing of weights, IFMs, and OFMs between the core data memory and the accelerator's scratchpad memory during the convolution operations. As shown in Table IX, the computing performance in terms of Ops/cycle of our extended RISC-V core (the core integrated with the accelerator) exceeds that of the extended core proposed in [19]. This improvement is due to the higher number of MAC operations

TABLE IX
CONVOLUTION ACCELERATOR COMPARISON WITH STATE-OF-THE-ART ASIC BASED CNN ACCELERATORS

Metrics \ SoA	EWS [17]	Chen et al. [19]	Gemmini [16], [25]	Sim et al. [12]	SNAP [14]	CONNA [15]	This Work	
							Accel. ^a	SoC ^b (Core+Accel.+Mem.)
Year	2024	2024	2019, 2021	2020	2021	2022	2025	
Accelerator Coupling	LC	TC	TC	Standalone	Standalone	TC	TC	
Host Core	RISC-V	RISC-V	RISC-V	NA	NA	RISC-V	RISC-V	
Bit-width	8-bit	8-bit	8-bit	10-bit	16-bit	8-bit	8-bit	
Frequency (MHz)	300	1000	500	60	33–480	200	150/300 ^c	
Num. of MACs	4096	128	256 16×16	512	252 (MULTs)	256	288	
Max. Throughput (Ops/Cycle)	8000 ^e	256 ^e	512 ^d 256 MAC Ops	1024 ^e	NA	512 ^e	1162^a	311^b
Max. Throughput (GOPS)	2400 ^e	256 ^e	256 ^d	61.44 ^e	NA	102.4 ^e	174.4 ^a	46.6 ^b

^a The throughput is based solely on the accelerator computation cycles, excluding the latency overhead of data transfer from/to the core data memory.

^b The throughput is based on the total execution time, including accelerator computation cycles and data transfer latency from/to the core data memory.

^c The core is running at 150 MHz, the accelerator is running at 300 MHz. The throughput is measured based on the core 150 MHz clock frequency.

^d GOPS is estimated based on the total number of MAC units for a 16×16 systolic array.

^e The reported performance is based on the accelerator computing performance. SoA Ops/Cycle is calculated based on reported values of (GOPS/Freq.).

TABLE X
CONVOLUTION ACCELERATOR COMPARISON WITH STATE-OF-THE-ART FPGA BASED CNN ACCELERATORS

Metrics \ SoA	Wang et al. [18]	Shao et al. [11]	Amoeba [13]	Light-OPU [8]	This Work	
					Accel. ^a	SoC ^b (Core+Accel.+Mem.)
Year	2023	2024	2024	2020	2025	
FPGA Device	Alveo-U200	VX690T	Arria 10	XCK325T	XCVCU9P	
Accelerator Coupling	TC	LC	Standalone	In-pipeline	TC	
Host Core	RISC-V	Host PC	-	Custom Pipeline	RISC-V	
Bit-width	8-bit	16-bit	8-bit	8-bit	8-bit	
Frequency (MHz)	73	200	200	200	150/300 ^c	
DSP Utilization	388	514	522	704	310	313
On-chip Mem.	1504 (BRAM)	483.5 (BRAM)	1036 (M20k)	193.5 (BRAM)	105 (URAM)	248 (URAM)
LUT/ALM Utilization	95.2K	139.5K	143K	173K	23K	28K
Max. Throughput (Ops/Cycle)	NA	NA	NA	NA	1162 ^a	311 ^b
Max. Throughput (GOPS)	75.7 ^e	153.14–199.8 ^e (single-core)	286.2 ^e	460.8 ^e	174.4 ^a	46.6 ^b
Static Power (mW)	NA	NA	NA	NA	2466	
Dynamic Power (mW)	NA	NA	NA	NA	960	1029
Total Power (mW)	3260	6931	8180	NA	3426	3495
Power Efficiency (GOPS/W) (Throughput / Dynamic Power)	NA	NA	NA	52.5 ^d	181.7^a	45.3^b
Power Efficiency (GOPS/W) (Throughput / Total Power)	23.2	22.09	35	NA	50.9^a	13.3^b

^a The throughput is based solely on the accelerator computation cycles, excluding the latency overhead of data transfer from/to the core data memory.

^b The throughput is based on the total execution time, including accelerator computation cycles and data transfer latency from/to the core data memory.

^c The core is running at 150 MHz, the accelerator is running at 300 MHz. The throughput is measured based on the core 150 MHz clock frequency.

^d The power efficiency is based on dynamic power as reported by [8]. ^e The reported performance is based on the accelerator computing performance.

supported by the convolution accelerator. The proposed accelerator consumes fewer DSP blocks in comparison with SoA accelerators targeting AMD/Xilinx FPGAs, as shown in Table X. An estimated total power consumption of 3495 mW was obtained for the complete SoC, which includes the extended RISC-V core with the tightly coupled accelerator as well as the associated core memory. The estimated total power consumption includes static and dynamic power is shown in Table X. In comparison with FPGA-based CNN

accelerators, our accelerator achieves a lower total power consumption than proposed accelerators in [11], [13]. As a result, our proposed accelerator achieves a power efficiency of 181.7 GOPS/W based on dynamic power consumption with a maximum computing throughput of 174.4 GOPS. The complete SoC, including the extended RISC-V core with memory latency overhead, achieves a maximum throughput of 46.6 GOPS and power efficiency of 13.3 GOPS/W based on total power consumption. As the accelerator is implemented

targeting an FPGA device, the maximum achievable frequency and computing performance are lower than ASIC-based accelerators. While the accelerator is designed for convolution workloads, future edge applications increasingly adopt models such as transformers, sparse, or mixed-precision operations. Our approach provides a flexible platform to support emerging workloads via custom instructions. Therefore, adapting the accelerator to new workloads requires modifying only the compute engine, while the existing decoder and load/store units inside the accelerator, which are responsible for managing interaction with the core via the XIF interface, can remain unchanged.

V. CONCLUSION

This work proposes ProCon-V, a programmable tightly coupled accelerator based on RISC-V custom instructions for CNN convolution operations. The proposed accelerator extends the RISC-V standard ISA to support normal 2-D convolution operations with different configurations for various CNN models. The accelerator is coupled to the CV32e40X RISC-V core through the extension interface to offload all convolution operations from the main pipeline execution unit. It supports the execution of multi-weight and multi-channel convolution layers with different stride configurations and kernel sizes. Moreover, the execution of convolution operations can be conducted directly over the RISC-V core through a set of custom instructions. The accelerator computing engine is based on a systolic array architecture consisting of 288 MAC units connected to a Psum management unit for OFM computation for multiple input channels. Also, the accelerator features a weight stationary dataflow management based on a hardware-supported on-the-fly Im2Col transformation for different stride and kernel size configurations. The accelerator is implemented targeting an AMD/Xilinx Virtex Ultrascale+ FPGA, with a maximum computing performance of 174.4 GOPS@INT8 and an efficiency of 181.7 GOPS/W. As for future work, it is planned to extend the current design to support multi-convolution accelerators based on mixed precision operations for higher computing performance.

REFERENCES

- [1] C. Hao et al., "Enabling design methodologies and future trends for edge AI: Specialization and codesign," *IEEE Des. Test; IEEE Des. Test Comput.*, vol. 38, no. 4, pp. 7–26, Aug. 2021, doi: 10.1109/MDAT.2021.3069952.
- [2] K. S. Zaman et al., "Custom hardware architectures for deep learning on portable devices: A review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6068–6088, Nov. 2022, doi: 10.1109/TNNLS.2021.3082304.
- [3] A. Garofalo et al., "XpulpNN: Accelerating quantized neural networks on RISC-V processors through ISA extensions," in *Proc. Des., Automat. & Test Eur. Conf. & Exhib. (DATE)*, Grenoble, France, 2020, pp. 186–191, doi: 10.23919/DAT48585.2020.9116529.
- [4] "NVDLA: NVIDIA deep learning accelerator." Accessed: Feb. 19, 2026. [Online]. Available: <https://github.com/nvdla/>
- [5] "AMD DPU for convolutional neural network." Accessed: Feb. 19, 2026. [Online]. Available: <https://www.xilinx.com/products/intellectual-property/dpu.html>
- [6] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Computer Archit. (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12, doi: 10.1145/3079856.3080246.
- [7] S. Liu et al., "Cambricon: An instruction set architecture for neural networks," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Computer Archit. (ISCA)*, Seoul, Korea (South), 2016, pp. 393–405, doi: 10.1109/ISCA.2016.42.
- [8] Y. Yu et al., "Light-OPU: An FPGA-based overlay processor for lightweight convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, New York, NY, USA: ACM, 2020, pp. 122–132, doi: 10.1145/3373087.3375311.
- [9] W. Zhao et al., "HIPU: A hybrid intelligent processing unit with fine-grained ISA for real-time deep neural network inference applications," *IEEE Trans. Very Large Scale Integr.*, vol. 31, no. 12, pp. 1980–1993, Dec. 2023, doi: 10.1109/TVLSI.2023.3327110.
- [10] D. Giri et al., "Ariane+ NVDLA: Seamless third-party IP integration with ESP," in *Proc. Workshop Computer Arch. Res. RISC-V (CARRV)*, 2020. Accessed: Feb. 19, 2026. [Online]. Available: https://carrv.github.io/2020/papers/CARRV2020_paper_14_Giri.pdf
- [11] Y. Shao et al., "A configurable accelerator for CNN-based remote sensing object detection on FPGAs," *IET Comput. Digit. Techn.*, vol. 1, 2024, Art. no. 4415342, doi: 10.1049/2024/4415342.
- [12] J. Sim et al., "An energy-efficient deep convolutional neural network inference processor with enhanced output stationary dataflow in 65-nm CMOS," *IEEE Trans. Very Large Scale Integr.*, vol. 28, no. 1, pp. 87–100, Jan. 2020, doi: 10.1109/TVLSI.2019.2935251.
- [13] X. Wu et al., "Amoeba: An efficient and flexible FPGA-based accelerator for arbitrary-Kernel CNNs," *IEEE Trans. Very Large Scale Integr.*, vol. 32, no. 6, pp. 1086–1099, Jun. 2024, doi: 10.1109/TVLSI.2024.3383871.
- [14] J.-F. Zhang et al., "SNAP: An efficient sparse neural acceleration processor for unstructured sparse deep neural network inference," *IEEE J. Solid-State Circuits*, vol. 56, no. 2, pp. 636–647, Feb. 2021, doi: 10.1109/JSSC.2020.3043870.
- [15] S.-S. Park and K.-S. Chung, "CONNA: Configurable matrix multiplication engine for neural network acceleration," *Electronics*, vol. 11, 2022, Art. no. 2373, doi: 10.3390/electronics11152373.
- [16] H. Genc et al., "Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures," 2019, *arXiv:1911.09925v1*.
- [17] C. Wang et al., "EWS: An energy-efficient CNN accelerator with enhanced weight stationary dataflow," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 71, no. 7, pp. 3478–3482, Jul. 2024, doi: 10.1109/TCSII.2024.3359511.
- [18] H. Wang et al., "Reconfigurable CNN accelerator embedded in instruction extended RISC-V core," in *Proc. Int. Conf. Electron. Technol. (ICET)*, Chengdu, China, 2023, pp. 945–954, doi: 10.1109/ICET58434.2023.10211513.
- [19] Y.-F. Chen et al., "Low DRAM memory access and flexible dataflow convolutional neural network accelerator based on RISC-V custom instruction," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Singapore, 2024, pp. 1–5, doi: 10.1109/ISCAS58744.2024.10558316.
- [20] "CV32e40X: OpenHW Group, "CV32e40X core," Accessed: Feb. 19, 2026. [Online]. Available: <https://github.com/openhwgroup/cv32e40x>
- [21] "OpenHW group specification: Core-V eXtension interface (CV-X-IF)," Accessed: Feb. 19, 2026. [Online]. Available: https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/x_ext.html
- [22] "RISC-V GNU toolchain." [Online]. Available: <https://github.com/riscv-collab/riscv-gnu-toolchain>
- [23] "AMD/Xilinx: FINN examples." Accessed: Feb. 19, 2026. [Online]. Available: <https://github.com/Xilinx/finn-examples>
- [24] A. Pullini et al., "Mr. Wolf: An energy-precision scalable parallel ultra low power SoC for IoT edge processing," *IEEE J. Solid-State Circuits*, vol. 54, no. 7, pp. 1970–1981, Jul. 2019, doi: 10.1109/JSSC.2019.2912307.
- [25] H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Des. Autom. Conf. (DAC)*, San Francisco, USA, 2021, pp. 769–774, doi: 10.1109/DAC18074.2021.9586216.
- [26] E. G. Cota et al., "An analysis of accelerator coupling in heterogeneous architectures," in *Proc. 52nd ACM/EDAC/IEEE Des. Autom. Conf. (DAC)*, San Francisco, CA, USA, 2015, pp. 1–6, doi: 10.1145/2744769.2744794.



Ahmed Kamaleldin (Member, IEEE) received the B.Sc. and M.Sc. degrees (Hons.) in electronics and electrical communications engineering from Cairo University, Egypt, in 2012 and 2017, respectively, and the Ph.D. (Dr.-Ing.) degree in computer science from TU Dresden, Germany, in 2023. He is currently a Postdoctoral Research Associate with the Adaptive Dynamic Systems (ADS) Chair, TU Dresden. His research interests include reconfigurable computing, multiprocessor systems-on-chip (MPSoC), domain-specific architectures (DSAs), hardware-software co-design, and heterogeneous systems-on-chip.



Habib Aouinti received the engineering degree (Dipl.-Ing.) from ABET-accredited École Polytechnique de Tunisie (EPT). He was awarded the DAAD Kospie Scholarship to conduct his master's thesis at the Chair of Adaptive Dynamic Systems (ADS), TU Dresden, Germany, in 2024.

Prior to studying at EPT, he ranked 7th out of 850 candidates in the national entrance exam for engineering schools in Tunisia, subsequently obtaining the Engineering Preparatory Cycle Degree.



Diana Göhringer (Member, IEEE) received the Ph.D. degree (*summa cum laude*) in electrical engineering and information technology from Karlsruhe Institute of Technology (KIT), Germany, in 2011. She is a Full Professor and holds the Chair of Adaptive Dynamic Systems, TU Dresden, Germany. From 2013 to 2017, she was an Assistant Professor and the Head of the MCA (application-specific multicore architectures) Research Group, Ruhr-University Bochum (RUB), Germany. Before that, she was working as the Head of the Young

Investigator Group CADEMA (Computer Aided Design and Exploration of Multicore Architectures), Institute for Data Processing and Electronics (IPE), Karlsruhe Institute of Technology (KIT). From 2007 to 2012, she was a Senior Scientist with Fraunhofer Institute of Optics, System Technologies and Image Exploitation IOSB, Ettlingen, Germany (formerly called FGAN-FOM). She is an author and a co-author of over 250 publications in international journals, conferences, and workshops. Additionally, she serves as a Technical Program Committee Member in several international conferences and workshops. She is a Reviewer and a Guest Editor of several international journals. Her research interests include reconfigurable computing, multiprocessor systems on-chip (MPSoCs), networks-on-chip, simulators/virtual platforms, hardware-software co-design, and runtime systems.